

操作手冊 — 每個範例的三條指令

這份是純操作。觀念看各週的 `README.md`，課程規劃看 [COURSE.md](#)。

先講清楚：`sim` 是什麼

`sim` 是我自己寫的批次檔 (`sim.bat`)，不是 Verilog 或 iverilog 的標準指令。教科書和網路上查不到它。它只是把下面三條包起來，順便幫你檢查結束碼。

你可以完全不用它。這份手冊裡給的都是原始的三條指令。

每次開始工作

到 `C:\Users\ROY.WANG\Desktop\VERI`，雙擊 `env.bat`。

提示字元要變成這樣才代表環境好了：

```
[OSS CAD Suite] C:\Users\ROY.WANG\Desktop\VERI>
```

[!WARNING] 前面一定要有 `[OSS CAD Suite]`。沒有就是環境沒載到，關掉重新雙擊 `env.bat`。不要用一般的 `cmd` 或 `PowerShell`。

三條指令的通則

```
iverilog -o sim.out <設計>.v <設計>_tb.v  
vvp sim.out  
gtkwave <設計>.gtkw
```

這一條	在做什麼
<code>iverilog -o sim.out A.v A_tb.v</code>	編譯。 <code>-o</code> 指定輸出檔名，後面接所有要編的 <code>.v</code>
<code>vvp sim.out</code>	模擬。這時候才會產生 <code>wave.vcd</code>
<code>gtkwave A.gtkw</code>	開波形。 <code>.gtkw</code> 是我預先配好訊號和顏色的設定檔

沒有 `.gtkw` 的話（例如你自己新寫的設計）就直接開 `vcd`：

```
gtkwave wave.vcd
```

然後自己在左邊挑訊號 → 按 `Insert` → 按 `Zoom Fit`。

★ 一定要加的第四條

```
echo %ERRORLEVEL%
```

打在 `iverilog` 後面，印 `0` 才是編譯成功。

[!IMPORTANT] Windows 上編譯失敗常常一個字都不印，看起來像成功了。這就是你一開始踩的坑。養成習慣：`iverilog` 之後就打這一行問一下。

檔名規則

我把所有範例都命名成同一套規則，所以三條指令長得都一樣：

檔案	是什麼
<code>01_gates.v</code>	設計檔
<code>01_gates_tb.v</code>	測試檔 (<code>_tb</code> = testbench)
<code>01_gates.gtkw</code>	波形設定
<code>sim.out</code>	編譯出來的執行檔 (跑完才有)
<code>wave.vcd</code>	模擬產生的波形資料 (跑完才有)

各週指令清單

照著複製貼上就能跑。每組四行（含檢查那行）。

第 0 週：語法入門 + 暖身

```
cd week00_warmup
```

軟體出身的人先讀 `week00_warmup\SYNTAX.md` —— 用 Python 對照講 Verilog 語法。

02_syntax — 語法練習場（`{}`、`for`、數字寫法）

```
iverilog -o sim.out 02_syntax.v 02_syntax_tb.v  
echo %ERRORLEVEL%  
vvp sim.out
```

01_counter — 4 位元計數器

```
iverilog -o sim.out 01_counter.v 01_counter_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave wave.gtkw
```

第 1 週：設計的基本概念

```
cd week01_basics
```

01_gates — 四個基本邏輯閘

```
iverilog -o sim.out 01_gates.v 01_gates_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 01_gates.gtkw
```

02_half_adder — 半加器

```
iverilog -o sim.out 02_half_adder.v 02_half_adder_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_half_adder.gtkw
```

03_full_adder — 全加器 ★

```
iverilog -o sim.out 03_full_adder.v 03_full_adder_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_full_adder.gtkw
```

第 2 週：資料型別與運算子

```
cd week02_types_ops
```

01_vectors — 向量與位元切片

```
iverilog -o sim.out 01_vectors.v 01_vectors_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_vectors.gtkw
```

02_operators — 五類運算子

```
iverilog -o sim.out 02_operators.v 02_operators_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_operators.gtkw
```

03_concat — 串接與符號延伸

```
iverilog -o sim.out 03_concat.v 03_concat_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_concat.gtkw
```

04_mux — 多工器

```
iverilog -o sim.out 04_mux.v 04_mux_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_mux.gtkw
```

05_xz — x 與 z ★

```
iverilog -o sim.out 05_xz.v 05_xz_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 05_xz.gtkw
```

06_operators_all — 運算子全集 (31 個全跑) 📖

```
iverilog -o sim.out 06_operators_all.v 06_operators_all_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 06_operators_all.gtkw
```

02_operators 每類只挑代表，這份把 / % ** -a ~^ || != > <= >= ===
!== ~&a ~|a ~^a <<< >>> 全部補齊，忘記哪個運算子是什麼就跑這個查。

第 3 週：組合與循序邏輯

```
cd week03_comb_seq
```

01_case_mux — case 多工器與解碼器

```
iverilog -o sim.out 01_case_mux.v 01_case_mux_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_case_mux.gtkw
```

02_latch_trap — latch 陷阱 ★

```
iverilog -o sim.out 02_latch_trap.v 02_latch_trap_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_latch_trap.gtkw
```

03_dff — 同步 / 非同步 reset

```
iverilog -o sim.out 03_dff.v 03_dff_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_dff.gtkw
```

04_blocking_vs_non — = vs <= ★★ 整套課程最重要

```
iverilog -o sim.out 04_blocking_vs_non.v 04_blocking_vs_non_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_blocking_vs_non.gtkw
```

05_shift_reg — 移位暫存器

```
iverilog -o sim.out 05_shift_reg.v 05_shift_reg_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 05_shift_reg.gtkw
```

06_comb_blocking — 組合邏輯用錯 <= 的後果 ★

```
iverilog -o sim.out 06_comb_blocking.v 06_comb_blocking_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 06_comb_blocking.gtkw
```

04 示範「時脈區塊用錯 = 」→ 正反器塌掉 06 示範「組合區塊用錯 <= 」→ 產生毛刺 兩個一起看才完整。

第 4 週：函式、參數、generate

```
cd week04_func_param
```

01_function

```
iverilog -o sim.out 01_function.v 01_function_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 01_function.gtkw
```

02_param_counter — 三個參數同時跑 ★

```
iverilog -o sim.out 02_param_counter.v 02_param_counter_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 02_param_counter.gtkw
```

03_generate — generate 做 N 位元加法器

```
iverilog -o sim.out 03_generate.v 03_generate_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 03_generate.gtkw
```

04_macro — 條件編譯

```
iverilog -o sim.out 04_macro.v 04_macro_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 04_macro.gtkw
```

第 5 週：功能模擬

```
cd week05_simulation
```

01_display_family — 四種印法 ★

```
iverilog -o sim.out 01_display_family.v 01_display_family_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_display_family.gtkw
```

02_self_check — 自我檢查測試平台 ★★

```
iverilog -o sim.out 02_self_check.v 02_self_check_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_self_check.gtkw
```

03_timescale

```
iverilog -o sim.out 03_timescale.v 03_timescale_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_timescale.gtkw
```

04_random_test — 亂數測試抓 bug

```
iverilog -o sim.out 04_random_test.v 04_random_test_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_random_test.gtkw
```

[!NOTE] 這個會印出一堆 [X] —— 正常的。它故意藏了一個 bug 讓你看測試怎麼抓到。

第 6 週：狀態機

```
cd week06_fsm
```

01_traffic_moore — 紅綠燈


```
iverilog -o sim.out 01_traffic_moore.v 01_traffic_moore_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_traffic_moore.gtkw
```

02_seq_detect — 序列偵測器

```
iverilog -o sim.out 02_seq_detect.v 02_seq_detect_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_seq_detect.gtkw
```

03_debounce — 按鍵去彈跳 ★★

```
iverilog -o sim.out 03_debounce.v 03_debounce_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_debounce.gtkw
```

04_vending — 投幣販賣機

```
iverilog -o sim.out 04_vending.v 04_vending_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_vending.gtkw
```

第 7 週：時序模擬

```
cd week07_timing_sim
```

01_gate_delay — 閘延遲

```
iverilog -o sim.out 01_gate_delay.v 01_gate_delay_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_gate_delay.gtkw
```

02_glitch — 毛刺 ★

```
iverilog -o sim.out 02_glitch.v 02_glitch_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_glitch.gtkw
```

03_cdc_sync — 雙 FF 同步器

```
iverilog -o sim.out 03_cdc_sync.v 03_cdc_sync_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_cdc_sync.gtkw
```

04_netlist — 閘級模擬 ★★ ★ 這個要多加參數

```
iverilog -gspecify -DICE40_HX -DNO_ICE40_DEFAULT_ASSIGNMENTS -o sim.out
04_netlist.v 04_netlist_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_netlist.gtkw
```

[!IMPORTANT] 只有這一個範例要加參數，因為它要模擬真實的 iCE40 元件時序：

- `-gspecify` → 啟用時序模型（沒有這個就沒有延遲，看不到毛刺）
- `-DICE40_HX` → 選用 iCE40-HX 系列的延遲數字
- `-DNO_ICE40_DEFAULT_ASSIGNMENTS` → 關掉元件庫的預設值

想重新綜合網表（改了 `_synth_src.v` 之後）：

```
yosys -p "read_verilog _synth_src.v; synth_ice40 -top logic_gate; write_verilog
-noattr 04_netlist_syn.v"
```

第 8 週：期中考

```
cd week08_midterm
```

01_seg7 — BCD 七段解碼器

```
iverilog -o sim.out 01_seg7.v 01_seg7_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

02_updown — 上下數計數器

```
iverilog -o sim.out 02_updown.v 02_updown_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

03_pattern_fsm — 密碼鎖

```
iverilog -o sim.out 03_pattern_fsm.v 03_pattern_fsm_tb.v
echo %ERRORLEVEL%
vvp sim.out
```

考試看的是終端機的分數，不一定要開波形。想看波形就再加 `gtkwave`
`01_seg7.gtkw`。

想跑參考解答（把設計檔換成 `_answers` 裡的）：

```
iverilog -o sim.out _answers\01_seg7.v 01_seg7_tb.v
vvp sim.out
```

第 9 週：記憶體

```
cd week09_memory
```

01_rom — ROM 與兩種讀取

```
iverilog -o sim.out 01_rom.v 01_rom_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_rom.gtkw
```

02_ram_sp — 單埠 RAM

```
iverilog -o sim.out 02_ram_sp.v 02_ram_sp_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_ram_sp.gtkw
```

03_ram_dp — 雙埠 RAM

```
iverilog -o sim.out 03_ram_dp.v 03_ram_dp_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_ram_dp.gtkw
```

04_fifo — FIFO ★★

```
iverilog -o sim.out 04_fifo.v 04_fifo_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_fifo.gtkw
```

其他

好用的小技巧

做什麼	怎麼做
叫回上一個指令	按 ↑ 上方向鍵 (最常用 · 改完 <code>.v</code> 重跑靠這個)
看目前資料夾有什麼	<code>dir</code>
只看 <code>.v</code> 檔	<code>dir *.v</code>
回上一層	<code>cd ..</code>
清畫面	<code>cls</code>
中斷卡住的模擬	<code>Ctrl + C</code>

GTKWave 操作

想做什麼	怎麼做
波形區一片空白	按工具列 Zoom Fit (左起第 5 個 · 方框四角往外那顆)
自己加訊號	左上角點模組名 → 下方選訊號 → 按左下 Insert
改數值格式	訊號名上按右鍵 → Data Format → Decimal / Hex
改顏色	右鍵 → Color Format
波形更好讀 ★	☰ → View → 勾 Show Filled High Values
加格線	☰ → View → 勾 Show Grid Lines
重跑後更新波形	☰ → File → Reload Waveform (訊號設定會保留)

[!TIP] 改了 .v 重跑之後 **GTKWave** 不用關。重跑完回 GTKWave 按 **File** → **Reload Waveform** 就好。

出問題時怎麼查

症狀	原因 / 解法
「不是內部或外部命令」	環境沒載到 → 關掉重新雙擊 env.bat
echo %ERRORLEVEL% 印很大的負數	PATH 缺 oss-cad-suite\lib → 一樣重開 env.bat
一堆英文錯誤，有 檔名.v:12:	你的 Verilog 語法錯， 12 是行號 (通常是上一行忘了分號)
波形一片紅色 x	reg 沒給初值 / reset 沒接 / 實例化漏接腳位
波形出現中間高度的線 z	wire 沒人驅動 / 三態沒開 enable
訊號卡住不動	latch → 組合 always 漏了 else 或 default
改了 .v 但波形沒變	忘記重跑了。三條要從第一條重跑
gtkwave 說找不到檔案	還沒跑 vvp ，或 cd 錯資料夾
Unknown module type: xxx	-o 後面漏了輸出檔名，把設計檔吃掉了 → 見下面 ⚠

[!CAUTION] `-o` 後面緊跟的那一個是「輸出檔名」，不是輸入檔。

```
iverilog -o 01_gates.v 01_gates_tb.v      ← X 少打 sim.out
~~~~~ 這個位置被當成輸出檔名
```

```
iverilog -o sim.out 01_gates.v 01_gates_tb.v  ← ✓ 四個字，不是三個
```

打錯的話 iverilog 只會讀到 testbench，找不到設計模組，就報 `Unknown module type` 加 `These modules were missing`。

編譯失敗時它不會寫輸出檔，所以設計檔通常還活著——但如果那次剛好編譯成功了，你的 `.v` 就會被執行檔蓋掉。看到這個錯誤先 `dir *.v` 確認檔案大小正常再繼續。

如果你想用捷徑

那三條打膩了的話，`sim` 就是它們的縮寫：

```
sim 01_gates
```

等同於前面那四行。加 `/nogui` 不開波形，不加參數會列出這個資料夾有哪些範例。另外 `weeks` 會列出所有週次的所有範例。

這兩個都是我寫的批次檔，不是標準工具，用不用隨你。